GUIA DE ESTUDO

SQL completo para iniciantes

Comandos essenciais de MySQL explicados com o exemplo do banco de restaurante: categorias, fornecedores, funcionários, clientes, produtos, estoque e pedidos.

Roteiro
Banco de dados → tabelas e regras → registros → consultas → alterações → relacionamentos → objetos úteis → transações.

1. O que é banco de dados e SQL?

Um banco de dados organiza informações. Uma tabela guarda registros em linhas e características em colunas. SQL é a linguagem usada para criar estruturas e trabalhar com os dados. No exemplo, tb_produtos guarda produtos e tb_pedidos registra pedidos.

2. Criar, selecionar, listar e apagar bancos

CREATE DATABASE cria um banco. USE seleciona qual banco receberá os próximos comandos. SHOW DATABASES lista os bancos disponíveis. DROP DATABASE remove o banco inteiro, incluindo suas tabelas e dados.

```
CREATE DATABASE db_drop_table_dieta;
USE db_drop_table_dieta;
SHOW DATABASES;
```

IF NOT EXISTS evita erro se o banco já existir; IF EXISTS evita erro se ele não existir.

```
CREATE DATABASE IF NOT EXISTS db_drop_table_dieta;
DROP DATABASE IF EXISTS db_drop_table_dieta;
```

Cuidado: DROP DATABASE apaga todo o conteúdo do banco. No seu script, essa linha serve para recomeçar do zero. Não a execute se quiser preservar os dados.

3. Criar tabelas, colunas e regras

CREATE TABLE define uma tabela. Tipos comuns: INT (inteiro), VARCHAR(n) (texto até n caracteres), CHAR(n) (texto de tamanho fixo), DECIMAL(p,s) (número decimal), DATE (data). NOT NULL exige valor. PRIMARY KEY identifica cada linha; UNIQUE impede repetição; CHECK valida uma condição.

```
CREATE TABLE tb_funcionarios (
  cod_funcionario INT NOT NULL PRIMARY KEY,
  nome VARCHAR(40) NOT NULL,
  cargo VARCHAR(40) NOT NULL,
  salario DECIMAL(10,2) NOT NULL CHECK (salario > 0)
);
```

Uma chave estrangeira conecta uma tabela a outra. Os valores relacionados precisam existir na tabela de referência.

```
CREATE TABLE tb_produtos (
  cod_produto INT NOT NULL PRIMARY KEY,
  nome_produto VARCHAR(60) NOT NULL,
  preco DECIMAL(8,2) NOT NULL,
  cod_categoria INT NOT NULL,
  cod_fornecedor INT NOT NULL,
  FOREIGN KEY (cod_categoria)
    REFERENCES tb_categorias(cod_categoria),
  FOREIGN KEY (cod_fornecedor)
    REFERENCES tb_fornecedores(cod_fornecedor)
);
```

A tabela de categorias e a de fornecedores devem ser criadas antes de produtos. Pedidos dependem de clientes e funcionários; itens_pedido depende de pedidos e produtos. Essa ordem também importa ao inserir registros.

Mais comandos para inspecionar tabelas

```
SHOW TABLES;
DESCRIBE tb_produtos;
SHOW CREATE TABLE tb_produtos;
```

DESCRIBE mostra colunas, tipos e chaves. SHOW CREATE TABLE exibe a definição SQL da tabela.

4. Inserir dados com INSERT

INSERT INTO adiciona registros. Indique as colunas para deixar claro a que posição pertence cada valor. Texto e datas ficam entre aspas simples.

```
INSERT INTO tb_categorias (cod_categoria, nome_categoria)
VALUES (1, 'Pizzas'),
       (2, 'Lanches'),
       (3, 'Combos');

INSERT INTO tb_produtos
  (cod_produto, nome_produto, preco, cod_categoria, cod_fornecedor)
VALUES (1, 'Pizza do Desespero', 49.90, 1, 1),
       (2, 'Hambúrguer do Pedro', 32.90, 2, 2);
```

Você também pode inserir um registro por vez. Para colunas opcionais, omita-as da lista ou use NULL se a coluna aceitar valor ausente.

```
INSERT INTO tb_estoque (cod_produto, quantidade, ultima_contagem)
VALUES (1, 12, '2026-11-02');
```

5. Consultar dados com SELECT

SELECT escolhe as colunas e FROM indica a tabela. O asterisco (*) representa todas as colunas.

```
SELECT nome_produto, preco
FROM tb_produtos;

SELECT *
FROM tb_clientes;
```

Filtre linhas com WHERE e operadores como =, <>, >, <, >= e <=. BETWEEN inclui os dois extremos; IN compara com uma lista; LIKE procura um padrão.

```
SELECT nome_produto, preco
FROM tb_produtos
WHERE preco BETWEEN 10 AND 40;

SELECT nome, cargo
FROM tb_funcionarios
WHERE cargo IN ('Garçom', 'Caixa');

SELECT nome
FROM tb_clientes
WHERE nome LIKE 'C%';
```

No LIKE, % representa qualquer sequência e _ representa um caractere. Use IS NULL para valores ausentes, por exemplo WHERE ultima_contagem IS NULL. DISTINCT remove duplicados.

6. Combinar condições, ordenar e limitar

```
SELECT nome, cidade
FROM tb_clientes
WHERE cidade = 'Brasília' AND sexo = 'M';

SELECT nome_produto AS Produto, preco AS valor
FROM tb_produtos
ORDER BY preco DESC
LIMIT 3;
```

AND exige que todas as condições sejam verdadeiras; OR aceita uma ou outra. ORDER BY organiza os resultados: ASC crescente (padrão), DESC decrescente. LIMIT mostra apenas uma quantidade definida. AS cria um apelido para uma coluna ou tabela.

7. Cálculos, agrupamentos e HAVING

COUNT conta linhas, SUM soma, AVG calcula média, MAX encontra o maior valor e MIN o menor. GROUP BY calcula por grupos; HAVING filtra os grupos depois do cálculo. WHERE filtra linhas antes do agrupamento.

```
SELECT cidade, COUNT(*) AS total_clientes
FROM tb_clientes
GROUP BY cidade;

SELECT cod_cliente, COUNT(*) AS total_pedidos
FROM tb_pedidos
GROUP BY cod_cliente
HAVING COUNT(*) > 1;
```

8. Relacionar tabelas com JOIN

JOIN combina linhas relacionadas; ON define a correspondência. JOIN (ou INNER JOIN) retorna linhas com correspondência nas duas tabelas. LEFT JOIN preserva todas as linhas da tabela à esquerda, mesmo sem correspondência à direita.

```
SELECT pe.num_pedido, c.nome AS cliente, f.nome AS atendente
FROM tb_pedidos AS pe
JOIN tb_clientes AS c
  ON pe.cod_cliente = c.cod_cliente
JOIN tb_funcionarios AS f
  ON pe.cod_funcionario = f.cod_funcionario
ORDER BY pe.num_pedido;
```

Os apelidos pe, c e f encurtam os nomes das tabelas.

9. Subconsultas e consultas de negócio

Uma subconsulta é um SELECT dentro de outro. Exemplo: encontrar produtos acima do preço médio.

```
SELECT nome_produto, preco
FROM tb_produtos
WHERE preco > (SELECT AVG(preco) FROM tb_produtos);
```

Exemplo baseado no seu banco: somar o valor dos itens comprados por cliente.

```
SELECT c.nome,
       SUM(i.quantidade * pr.preco) AS total_gasto
FROM tb_clientes AS c
JOIN tb_pedidos AS pe ON c.cod_cliente = pe.cod_cliente
JOIN tb_itens_pedido AS i ON pe.num_pedido = i.num_pedido
JOIN tb_produtos AS pr ON i.cod_produto = pr.cod_produto
GROUP BY c.nome
ORDER BY total_gasto DESC;
```

10. Alterar e apagar dados

UPDATE altera valores; DELETE apaga linhas. Sempre confira o WHERE: sem ele, o comando pode atingir todos os registros.

```
UPDATE tb_produtos
SET preco = 35.90
WHERE cod_produto = 2;

DELETE FROM tb_produtos
WHERE cod_produto = 2;
```

Confira quais linhas seriam atingidas usando um SELECT com a mesma condição antes de executar a alteração ou exclusão.

Alterar a estrutura com ALTER TABLE

```
ALTER TABLE tb_produtos
ADD descricao VARCHAR(200);

ALTER TABLE tb_produtos
DROP COLUMN descricao;
```

ALTER TABLE modifica a estrutura: pode adicionar, alterar ou remover colunas e restrições. DROP COLUMN remove os dados daquela coluna.

11. Views, índices e transações

Uma VIEW guarda uma consulta com nome para reutilizá-la como se fosse uma tabela. Um índice pode acelerar buscas em colunas pesquisadas com frequência, mas ocupa espaço e pode deixar inserções/alterações um pouco mais lentas.

```
CREATE VIEW vw_produtos_categoria AS
SELECT pr.nome_produto, ca.nome_categoria, pr.preco
FROM tb_produtos AS pr
JOIN tb_categorias AS ca
  ON pr.cod_categoria = ca.cod_categoria;

SELECT * FROM vw_produtos_categoria;

CREATE INDEX idx_produtos_nome
ON tb_produtos (nome_produto);
```

Transações agrupam alterações. COMMIT confirma; ROLLBACK desfaz alterações ainda não confirmadas. A disponibilidade e o efeito dependem do mecanismo de armazenamento da tabela, normalmente InnoDB no MySQL.

```
START TRANSACTION;
UPDATE tb_estoque
SET quantidade = quantidade - 1
WHERE cod_produto = 1;
COMMIT;
-- Use ROLLBACK em vez de COMMIT para desfazer.
```

12. Função, procedure e trigger

Função retorna um valor e pode ser usada numa expressão. Procedure executa uma sequência de comandos e é chamada com CALL. Trigger é executada automaticamente quando ocorre INSERT, UPDATE ou DELETE na tabela configurada.

```
DELIMITER $$
CREATE FUNCTION aumentoPreco(preco_atual DECIMAL(8,2))
RETURNS DECIMAL(8,2)
DETERMINISTIC
BEGIN
  RETURN preco_atual * 1.10;
END $$
DELIMITER ;

SELECT aumentoPreco(20.00);
```

No MySQL Workbench, selecione o script e use o botão de execução. DELIMITER muda temporariamente o marcador de fim de comando para permitir os ponto e vírgulas dentro do bloco BEGIN...END. A função do exemplo aumenta um preço em 10%.

13. Procedure e trigger: leitura do exemplo

```
DELIMITER $$
CREATE PROCEDURE InserirCliente(
  IN p_cod INT,
  IN p_nome VARCHAR(60),
  IN p_cidade VARCHAR(40),
  IN p_sexo CHAR(1)
)
BEGIN
  INSERT INTO tb_clientes (cod_cliente, nome, cidade, sexo)
  VALUES (p_cod, p_nome, p_cidade, p_sexo);
END $$
DELIMITER ;

CALL InserirCliente(6, 'Valdemar', 'Brasília', 'M');
```

Para criar uma trigger, defina o evento, o momento (BEFORE ou AFTER) e o que fazer. NEW representa os novos valores; OLD representa os valores anteriores em UPDATE/DELETE. Exemplo ilustrativo: registrar mudanças de preço em uma tabela de auditoria.

```
DELIMITER $$
CREATE TRIGGER antes_de_alterar_preco
BEFORE UPDATE ON tb_produtos
FOR EACH ROW
BEGIN
  IF NEW.preco < 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Preco nao pode ser negativo';
  END IF;
END $$
DELIMITER ;
```

Esse exemplo rejeita preço negativo antes de atualizar. Triggers são automáticas; para uma regra simples também avalie CHECK ou validação da aplicação.

14. Ordem prática para executar seu projeto

Passo	O que fazer
1	Criar/selecionar o banco; só apagar com DROP se quiser reiniciar.
2	Criar tabelas na ordem das chaves estrangeiras.
3	Inserir primeiro os dados referenciados e depois os dependentes.
4	Executar SELECTs para conferir os dados.
5	Testar UPDATE/DELETE com WHERE e confirmar as linhas antes.
6	Criar views, rotinas e triggers apenas quando entender a regra necessária.

Resumo rápido: CREATE cria; SHOW lista; USE seleciona; INSERT adiciona; SELECT consulta; UPDATE altera; DELETE apaga linhas; ALTER modifica a estrutura; DROP remove um objeto.